

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

Д.О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №6

РЕАЛИЗАЦИЯ ПОЛУАВТОМАТИЧЕСКИХ И
АВТОМАТИЧЕСКИХ РЕЖИМОВ УПРАВЛЕНИЯ
ОБОРУДОВАНИЕМ

по курсу: Интернет вещей

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4326

подпись, дата

Г. С. Томчук

инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Приобретение навыков ограничения получения данных и приведения их к необходимому формату.

2 Задание

Наладить обмен данными между оборудованием внутри системы, создать условия переключения состояния оборудования. Взаимодействие элементов системы должно соответствовать предложенному в первой лабораторной работе проекту.

Вариант задания - Умная теплица.

3 Выполнение задания

Были внесены изменения в программный код системы умной теплицы, направленные на реализацию автоматического и полуавтоматического управления устройствами на основе показаний датчиков. Первым шагом стала модификация датчиков: в класс `Sensor` добавлена эмуляция изменений измеряемых величин. Метод `measure()` генерирует случайные значения в реалистичных диапазонах (температура 15–35 °C, влажность воздуха 30–80 %, влажность почвы 20–60 %), а метод `connect()` автоматически вызывает `measure()` при каждом опросе датчика, если отсутствует управляющая команда. Это позволило имитировать реальные изменения среды. На рисунках 1-2 – пример генерации случайных значений, отображаемых в веб-интерфейсе.

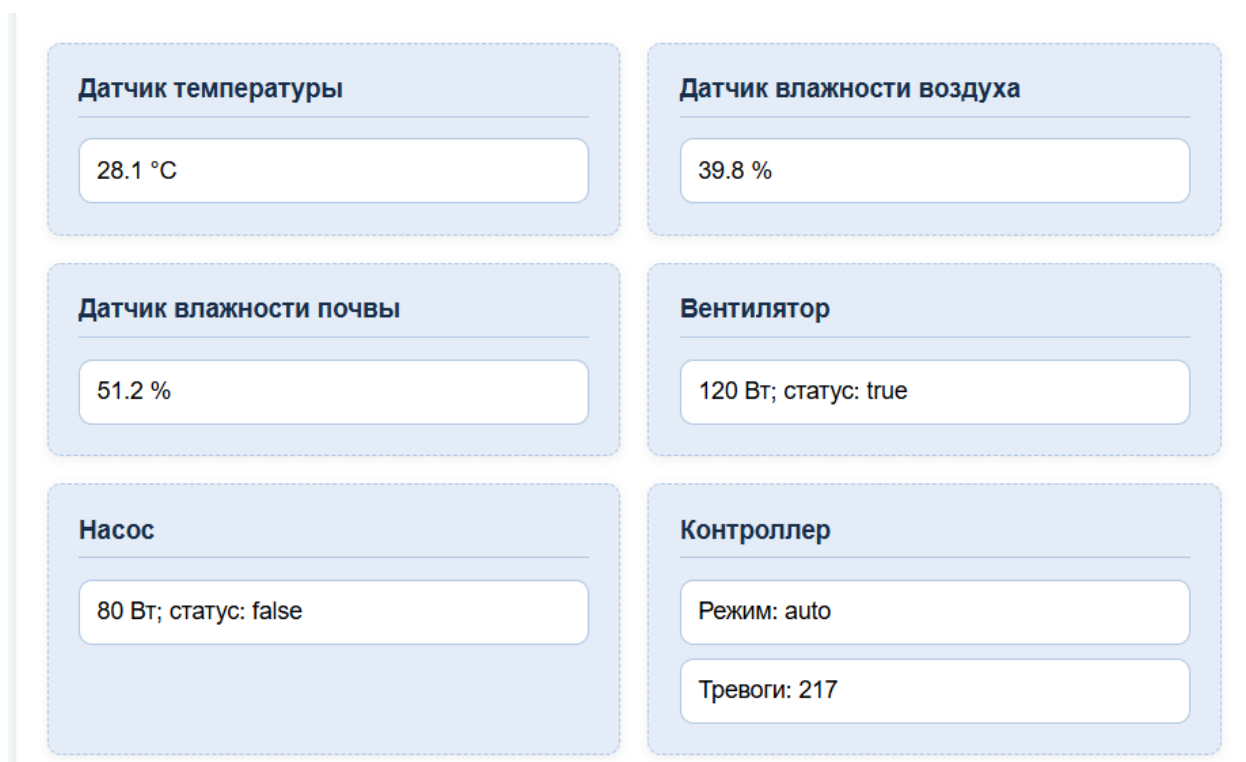


Рисунок 1 – Генерация значений в веб-интерфейсе и реализация автоматического управления

Датчик температуры 19 °C	Датчик влажности воздуха 56.9 %
Датчик влажности почвы 39.4 %	Вентилятор 120 Вт; статус: false
Насос 80 Вт; статус: false	Контроллер Режим: auto Тревоги: 228

Рисунок 2 – Генерация значений в веб-интерфейсе и реализация автоматического управления

На рисунке 3 представлен код функции `measure()`, а на рисунке 4 – логи сервера.

```
def measure(self): 1 usage  1 Kirill_Cheremushkin *
# Генерация случайных значений
if self.type == "temperature":
    self.value = round(random.uniform(a: 15.0, b: 35.0), 1)
elif self.type == "humidity":
    self.value = round(random.uniform(a: 30.0, b: 80.0), 1)
elif self.type == "soil_moisture":
    self.value = round(random.uniform(a: 20.0, b: 60.0), 1)
else:
    self.value = round(random.uniform(a: 0, b: 100), 1)
print(f"[LOG] Датчик {self.name} (ID: {self.id}) измерил: {self.value} {self.unit}")
return self.value
```

Рисунок 3 – Метод `measure()`

```
[LOG] Опрос датчика Датчик влажности почвы: текущее значение 56.1 %
[LOG] Подключение к БД выполнено, записей за сегодня: 0, путь: ./data
[LOG] Датчик Датчик влажности почвы (ID: 3) измерил: 43.2 %
[LOG] Датчик Датчик влажности воздуха (ID: 2) измерил: 54.5 %[LOG] Опрос датчика Датчик влажности почвы: текущее значение 43.2 %
[LOG] Опрос датчика Датчик температуры: текущее значение 32.3 °C
[LOG] Опрос исполнительного устройства Вентилятор: 120.0 Вт, статус: False
```

Рисунок 4 – Логи сервера

Для организации автоматического управления в классе MainControlUnit были добавлены ссылки на исполнительные устройства (вентилятор и насос) при инициализации, а также реализован новый метод auto_control(temperature, soil_moisture). Данный метод проверяет текущий режим контроллера: если режим “auto”, то в зависимости от полученных показаний принимаются решения.

При температуре воздуха выше 28 С вентилятор включается, при снижении до 28 С и ниже – выключается. Аналогично, при влажности почвы менее 30 % включается насос, при достижении 30 % и выше – выключается. Все автоматические действия фиксируются в списке тревог (alerts) и выводятся в консоль сервера. На рисунке 5 представлен метод auto_control.

```
def auto_control(self, temperature: float, soil_moisture: float): 2 usages new *
    """Автоматическое управление на основе показаний датчиков"""
    if self.mode != "auto":
        print("[LOG] Автоматика отключена (ручной режим)")
        return

    # Управление вентилятором по температуре
    if self.fan:
        if temperature > 28.0 and not self.fan.status:
            self.fan.turn_on()
            self.alerts.append(f"Авто: вентилятор включён при {temperature}°C")
            print(f"[LOG] Автоматика: включён вентилятор (температура {temperature}°C)")
        elif temperature <= 28.0 and self.fan.status:
            self.fan.turn_off()
            self.alerts.append(f"Авто: вентилятор выключен при {temperature}°C")
            print(f"[LOG] Автоматика: выключен вентилятор (температура {temperature}°C)")

    # Управление насосом по влажности почвы
    if self.pump:
        if soil_moisture < 30.0 and not self.pump.status:
            self.pump.turn_on()
            self.alerts.append(f"Авто: насос включён (влажность почвы {soil_moisture}%)")
            print(f"[LOG] Автоматика: включён насос (влажность почвы {soil_moisture}%)")
        elif soil_moisture >= 30.0 and self.pump.status:
            self.pump.turn_off()
            self.alerts.append(f"Авто: насос выключен (влажность почвы {soil_moisture}%)")
            print(f"[LOG] Автоматика: выключен насос (влажность почвы {soil_moisture}%)")
```

Рисунок 5 – Метод auto_control

Интеграция автоматики в маршруты веб-приложения выполнена в файле app.py. После создания объектов датчиков и устройств контроллеру были переданы ссылки на вентилятор и насос. Добавлены глобальные переменные last_temperature и last_soil (рисунок 6) для хранения последних

актуальных показаний. В маршрутах /connect/temperature и /connect/soil после получения данных от датчика обновляется соответствующая переменная и вызывается controller.auto_control(), что обеспечивает оперативное реагирование на изменение условий. На рисунке 7 показаны изменённые маршруты.

```
last_temperature = 0.0
last_soil = 0.0
```

Рисунок 6 – Новые глобальные переменные

```
@app.route("/connect/temperature")  Grigory Tomchuk *
def connect_temperature():
    global last_temperature
    data = temperature_sensor.connect()
    last_temperature = data["value"]
    controller.auto_control(last_temperature, last_soil)
    return jsonify(data)

@app.route("/connect/soil")  Grigory Tomchuk *
def connect_soil():
    global last_soil
    data = soil_sensor.connect()
    last_soil = data["value"]
    controller.auto_control(last_temperature, last_soil)
    return jsonify(data)
```

Рисунок 7 – Изменённые маршруты

Для реализации полуавтоматического режима в веб-интерфейсе уже имелся выпадающий список для переключения режима контроллера (controller_mode_cmd). При отправке команды значение сохраняется в объекте контроллера. В методе auto_control присутствует проверка if self.mode != "auto": return, поэтому при ручном режиме автоматическое управление не выполняется, и пользователь может полностью управлять устройствами через панель управления. Это позволяет оператору при необходимости брать

управление на себя. На рисунке 8 представлен веб-интерфейс с выбранным ручным режимом, а также перешедшие в полуавтоматический режим актуаторы.

Датчик влажности почвы	Вентилятор	Насос	Контроллер
Питание	Мощность, Вт	Мощность, Вт	Режим
Выкл	6	26	Ручной
Калибровка, %	Питание	Питание	☒ Сбросить тревоги
42	Вкл	Вкл	
	Действие (лог)	Действие (лог)	
	ускорить	полив	

Рисунок 8 – Веб-интерфейс

Данные насоса и вентилятора теперь не изменяются со временем, а устанавливаются вручную. Примеры на рисунках 9-10.

Датчик температуры 34.3 °C	Датчик влажности воздуха 62.7 %
Датчик влажности почвы 36.8 %	Вентилятор 6 Вт; статус: true
Насос 26 Вт; статус: true	Контроллер Режим: manual Тревоги: 0

Рисунок 9 – Отсутствие изменений у вентилятора и насоса

Агроном (оператор)	
Датчик температуры 20.8 °C	Датчик влажности воздуха 59.5 %
Датчик влажности почвы 57.3 %	Вентилятор 6 Вт; статус: true
Насос 26 Вт; статус: true	Контроллер Режим: manual Тревоги: 0

Рисунок 10 – Отсутствие изменений у вентилятора и насоса

Листинг программы представлен в Приложениях к лабораторной работе.

Вывод

В ходе выполнения лабораторной работы были успешно реализованы автоматический и полуавтоматический (ручной) режимы управления оборудованием умной теплицы. Для этого в класс датчиков добавлена эмуляция динамического изменения показаний (температура, влажность воздуха, влажность почвы), что позволило моделировать реальные условия окружающей среды без использования физического оборудования. В контроллере реализована логика автоматического управления: при превышении температуры порога 28 С включается вентилятор, при снижении ниже этого значения – выключается; при падении влажности почвы ниже 30 % запускается насос, при нормализации – отключается. Все автоматические действия фиксируются в логе сервера.

Налажен обмен данными между датчиками и устройствами через центральный контроллер. Показания датчиков передаются в контроллер, который на их основе принимает решения и воздействует на актуаторы. Полуавтоматический режим реализован путём добавления возможности переключения режима контроллера (auto/manual) через веб-интерфейс. В ручном режиме автоматика отключается, и оператор может самостоятельно управлять вентилятором и насосом, отправляя команды через панель управления. Это позволяет реагировать на нештатные ситуации или при необходимости отменить автоматическое поведение.

Тестирование подтвердило корректную работу обоих режимов: в автоматическом режиме устройства переключаются согласно заданным пороговым значениям, в ручном – все команды выполняются только по инициативе пользователя. Веб-интерфейс в реальном времени отображает актуальные показания датчиков, состояние устройств и текущий режим контроллера, что делает управление интуитивно понятным.

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А

Листинг app.py

```
import logging

from flask import Flask, jsonify, render_template, request

from devices import Actuator, Database, MainControlUnit, Sensor

app = Flask(__name__)

temperature_sensor = Sensor(1, "Датчик температуры", "temperature", "°C",
"temperature")
humidity_sensor = Sensor(2, "Датчик влажности воздуха", "humidity", "%",
"humidity")
soil_sensor = Sensor(3, "Датчик влажности почвы", "soil_moisture", "%", "soil")
fan = Actuator(101, "Вентилятор", "fan", 120.0)
pump = Actuator(102, "Насос полива", "pump", 80.0)
controller = MainControlUnit(fan_device=fan, pump_device=pump)
database = Database("sqlite:///greenhouse.db", "./data")

last_temperature = 0.0
last_soil = 0.0

temperature_sensor.turn_on()
humidity_sensor.turn_on()
soil_sensor.turn_on()
fan.turn_on()
pump.turn_on()

@app.route("/")
def index():
    return render_template("dashboard.html")

@app.route("/connect/temperature")
def connect_temperature():
    global last_temperature
    data = temperature_sensor.connect()
    last_temperature = data["value"]
    controller.auto_control(last_temperature, last_soil)
    return jsonify(data)
```

```

@app.route("/connect/soil")
def connect_soil():
    global last_soil
    data = soil_sensor.connect()
    last_soil = data["value"]
    controller.auto_control(last_temperature, last_soil)
    return jsonify(data)

@app.route("/connect/humidity")
def connect_humidity():
    return jsonify(humidity_sensor.connect())

@app.route("/connect/fan")
def connect_fan():
    return jsonify(fan.connect())

@app.route("/connect/pump")
def connect_pump():
    return jsonify(pump.connect())

@app.route("/connect/controller")
def connect_controller():
    return jsonify(controller.connect())

@app.route("/connect/database")
def connect_database():
    return jsonify(database.connect())

@app.route("/connect/command", methods=["GET"])
def connect_command():
    """Применить управляющие параметры ко всем вещам."""
    return jsonify(
        {
            "temperature": temperature_sensor.connect(request),
            "humidity": humidity_sensor.connect(request),
            "soil": soil_sensor.connect(request),
            "fan": fan.connect(request),
            "pump": pump.connect(request),
            "controller": controller.connect(request),
            "database": database.connect(request),
        }
    )

```

```
if __name__ == "__main__":
    logging.getLogger("werkzeug").setLevel(logging.ERROR)
    app.run(debug=True)
```

ПРИЛОЖЕНИЕ А

Листинг devices.py

```
import random
import re
from abc import ABC, abstractmethod
from datetime import datetime
from typing import Dict, List, Optional, Union

from werkzeug.wappers import Request

class Device(ABC):
    def __init__(self, device_id: int, name: str, status: bool = False):
        self.id = device_id
        self.name = name
        self.status = status

    def turn_on(self):
        self.status = True
        print(f'[LOG] {self.name} (ID: {self.id}) включён')

    def turn_off(self):
        self.status = False
        print(f'[LOG] {self.name} (ID: {self.id}) выключен')

    def get_status(self) -> bool:
        print(f'[LOG] Запрос статуса {self.name} (ID: {self.id}) -> {self.status}')
        return self.status

    @abstractmethod
    def connect(
        self, request: Optional[Request] = None
    ) -> Dict[str, Union[int, float, str, bool]]:
        """Мониторинг (request is None) или применение команд (передан request)."""

class Sensor(Device):
    def __init__(
        self,
        device_id: int,
        name: str,
        sensor_type: str,
```

```

    unit: str,
    control_prefix: str,
):
    super().__init__(device_id, name)
    self.type = sensor_type
    self.unit = unit
    self.control_prefix = control_prefix
    self.value: float = 0.0

def measure(self):
    # Генерация случайных значений
    if self.type == "temperature":
        self.value = round(random.uniform(15.0, 35.0), 1)
    elif self.type == "humidity":
        self.value = round(random.uniform(30.0, 80.0), 1)
    elif self.type == "soil_moisture":
        self.value = round(random.uniform(20.0, 60.0), 1)
    else:
        self.value = round(random.uniform(0, 100), 1)
    print(f'[LOG] Датчик {self.name} (ID: {self.id}) измерил: {self.value} {self.unit}')
    return self.value

def calibrate(self, value: float):
    self.value = value
    print(
        f'[LOG] Датчик {self.name} (ID: {self.id}) откалиброван на значение {value} {self.unit}'
    )

def _apply_control(self, request: Request) -> None:
    p = self.control_prefix
    power_raw = request.args.get(f'{p}_power', "").strip().lower()
    if power_raw:
        # Допустимо on, off, 1, 0, true, false
        if re.match(r'^(on|off|1|0|true|false)$', power_raw):
            if power_raw in ("on", "1", "true"):
                self.turn_on()
            else:
                self.turn_off()
        else:
            print(
                f'[LOG] Ошибка: недопустимое значение power '{power_raw}' для {self.name}. Допустимо: on/off/1/0/true/false"
            )

```

```

raw_cal = request.args.get(f'{p}_calibrate', "").strip()
if raw_cal:
    try:
        # Заменяем запятую на точку и преобразуем в float
        value = float(raw_cal.replace(",", "."))
        self.calibrate(value)
    except ValueError:
        print(f"[LOG] Ошибка: калибровка {self.name} – некорректное число '{raw_cal}'")

```

```

def connect(self, request: Optional[Request] = None) -> Dict[str, Union[int, float, str, bool]]:
    if request is not None:
        self._apply_control(request)
        print(
            f"[LOG] Управляющая команда для датчика {self.name}: значение {self.value} {self.unit}, статус {self.status}")
    else:
        # Без команд выполняем измерение
        self.measure()
        print(f"[LOG] Опрос датчика {self.name}: текущее значение {self.value} {self.unit}")
    return {
        "id": self.id,
        "name": self.name,
        "type": self.type,
        "unit": self.unit,
        "value": self.value,
        "status": self.status,
    }

```

```

class Actuator(Device):
    def __init__(
        self, device_id: int, name: str, actuator_type: str, power: float = 0.0
    ):
        super().__init__(device_id, name)
        self.type = actuator_type
        self.power = power
        self.control_prefix = actuator_type

    def apply_action(self, action: str):
        print(
            f"[LOG] Исполнительное устройство {self.name} (ID: {self.id}) выполняет действие: {action}"
        )

```

```

def set_power(self, level: float):
    self.power = level
    print(
        f'[LOG] Мощность устройства {self.name} (ID: {self.id}) установлена
на {level} Вт"
    )

def _apply_control(self, request: Request) -> None:
    p = self.control_prefix

    power_kw = request.args.get(f'{p}_power_cmd', "").strip()
    if power_kw:
        try:
            self.set_power(float(power_kw.replace(",", ".")))
        except ValueError:
            print(f'[LOG] Ошибка: мощность {self.name} – некорректное число
'{power_kw}''')

    action = request.args.get(f'{p}_action', "").strip()
    if action:
        if re.match(r'^[a-zA-Za-яA-Я0-9_-]+$', action):
            self.apply_action(action)
        else:
            print(
                f'[LOG] Ошибка: действие '{action}' для {self.name} содержит
недопустимые символы. Разрешены буквы, цифры, '_', '-'')

    st = request.args.get(f'{p}_power_state', "").strip().lower()
    if st:
        if re.match(r'^(on|off|1|0|true|false)$', st):
            if st in ("on", "1", "true"):
                self.turn_on()
            else:
                self.turn_off()
        else:
            print(f'[LOG] Ошибка: неверное состояние питания '{st}' для
{self.name}')

def connect(
    self, request: Optional[Request] = None
) -> Dict[str, Union[int, float, str, bool]]:
    if request is not None:
        self._apply_control(request)
    print(
        f'[LOG] Управление {self.name}: мощность {self.power} Вт, статус

```

```

{self.status}"
    )
    else:
        print(
            f"[LOG] Опрос исполнительного устройства {self.name}:
{self.power} Вт, статус: {self.status}"
        )
    return {
        "id": self.id,
        "name": self.name,
        "type": self.type,
        "power": self.power,
        "status": self.status,
    }

class MainControlUnit:
    def __init__(self, fan_device=None, pump_device=None):
        self.mode = "auto"
        self.schedule: Dict[str, str] = {}
        self.alerts: List[str] = []
        self.fan = fan_device
        self.pump = pump_device

    def process_data(self, sensor_data: Dict[int, float]):
        print(f"[LOG] Контроллер обрабатывает данные с датчиков:
{sensor_data}")

    def send_command(self, actuator: Actuator, command: str):
        print(
            f"[LOG] Контроллер отправляет команду '{command}' устройству
{actuator.name} (ID: {actuator.id})"
        )
        actuator.apply_action(command)

    def check_alerts(self):
        print("[LOG] Контроллер проверяет наличие тревог...")
        if self.alerts:
            print(f"[LOG] Активные тревоги: {self.alerts}")
        else:
            print("[LOG] Тревог нет")

    def _apply_control(self, request: Request) -> None:
        mode = request.args.get("controller_mode", "").strip().lower()
        if mode:
            if re.match(r'^(auto|manual)$', mode):

```



```

        self.mode = mode
        print(f"[LOG] Контроллер: режим установлен в {self.mode}")
    else:
        print(f"[LOG] Ошибка: неверный режим '{mode}'. Допустимо: auto /
manual")

    clear = request.args.get("controller_clear_alerts", "").strip().lower()
    if clear:
        if re.match(r'^(1|true|yes|on)$', clear):
            self.alerts = []
            print("[LOG] Контроллер: список тревог очищен")
        else:
            print(f"[LOG] Ошибка: неверное значение clear_alerts '{clear}'.
Игнорируем.")

    def connect(
        self, request: Optional[Request] = None
    ) -> Dict[str, Union[str, int, List[str]]]:
        if request is not None:
            self._apply_control(request)
        print(
            f"[LOG] Подключение к контроллеру выполнено, режим: {self.mode},
количество тревог: {len(self.alerts)}"
        )
        return {
            "mode": self.mode,
            "alerts_count": len(self.alerts),
            "alerts": list(self.alerts),
        }

    def auto_control(self, temperature: float, soil_moisture: float):
        """Автоматическое управление на основе показаний датчиков"""
        if self.mode != "auto":
            print("[LOG] Автоматика отключена (ручной режим)")
            return

        # Управление вентилятором по температуре
        if self.fan:
            if temperature > 28.0 and not self.fan.status:
                self.fan.turn_on()
                self.alerts.append(f"Авто: вентилятор включён при {temperature}°C")
                print(f"[LOG] Автоматика: включён вентилятор (температура
{temperature}°C)")
            elif temperature <= 28.0 and self.fan.status:
                self.fan.turn_off()

```

```
        self.alerts.append(f"Авто: вентилятор выключен при  
{temperature}°C")  
        print(f"[LOG] Автоматика: выключен вентилятор (температура  
{temperature}°C)")
```

```
# Управление насосом по влажности почвы  
if self.pump:  
    if soil_moisture < 30.0 and not self.pump.status:  
        self.pump.turn_on()  
        self.alerts.append(f"Авто: насос включён (влажность почвы  
{soil_moisture}%)")  
        print(f"[LOG] Автоматика: включён насос (влажность почвы  
{soil_moisture}%)")  
    elif soil_moisture >= 30.0 and self.pump.status:  
        self.pump.turn_off()  
        self.alerts.append(f"Авто: насос выключен (влажность почвы  
{soil_moisture}%)")  
        print(f"[LOG] Автоматика: выключен насос (влажность почвы  
{soil_moisture}%)")
```

```
class Database:
```

```
    def __init__(self, connection_string: str, storage_path: str):  
        self.connection_string = connection_string  
        self.storage_path = storage_path  
        self.records_today: int = 0
```

```
    def save_reading(self, sensor_id: int, value: float, timestamp: datetime):  
        self.records_today += 1  
        print(  
            f"[LOG] БД: сохранено показание датчика {sensor_id} = {value} в  
{timestamp}"  
        )
```

```
    def save_event(self, device_id: int, action: str, timestamp: datetime):  
        self.records_today += 1  
        print(  
            f"[LOG] БД: сохранено событие устройства {device_id}: {action} в  
{timestamp}"  
        )
```

```
    def get_history(  
        self, start: datetime, end: datetime, device_id: Optional[int] = None  
    ):  
        print(  
            f"[LOG] БД: получение истории за период {start} - {end} для устройства {device_id}"  
        )
```

```
        f"[LOG] БД: запрос истории с {start} по {end} для устройства  
{device_id if device_id else 'всех'}"  
    )
```

```
    return []
```

```
def _apply_control(self, request: Request) -> None:  
    cmd = request.args.get("database_command", "").strip()  
    if cmd:  
        if re.match(r'^[a-zA-Z0-9_\-]+$' , cmd):  
            print(f"[LOG] БД: принята управляющая команда '{cmd}'")  
        else:  
            print(  
                f"[LOG] Ошибка: команда БД '{cmd}' содержит недопустимые  
символы. Разрешены буквы, цифры, '_', '-'")
```

```
def connect(self, request: Optional[Request] = None) -> Dict[str, Union[str,  
int]]:  
    if request is not None:  
        self._apply_control(request)  
    data = {  
        "connection": "ok",  
        "storage_path": self.storage_path,  
        "records_today": self.records_today,  
    }  
    print(  
        f"[LOG] Подключение к БД выполнено, записей за сегодня:  
{data['records_today']}, путь: {data['storage_path']}"  
    )  
    return data
```